Motor de Análisis Técnico de Acciones

Evaluación determinista + IA solo interpretativa

Rubén Touceda · Máster Desarrollo con IA (Big School) · Octubre 2026

Proyecto real de uso personal: seleccionar acciones para el análisis fundamental.

El problema

- Filtraba acciones en Finviz por parámetros, pero luego tocaba ir **una a una** clasificándolas como más o menos interesantes
- Ese triaje manual no escala: decenas de candidatas pidiendo **análisis fundamental** (históricos, beneficios, ganancias)
- Faltaba un **prefiltro cuantitativo**: que cada acción se evalúe sola con ratio riesgo/beneficio + entrada + stop-loss…
- …para quedarme solo con las mejores y dedicarles el análisis fundamental

La idea: división clara

1. **Motor determinista**: reglas técnicas → score 0-100 + R:R + cumplimiento
2. **IA solo interpretativa**: redacta la síntesis desde métricas ya calculadas
3. **Uso de API externas**: Finviz, Polygon, Finnhub, OpenRouter todas sin coste

¿Cómo funciona?

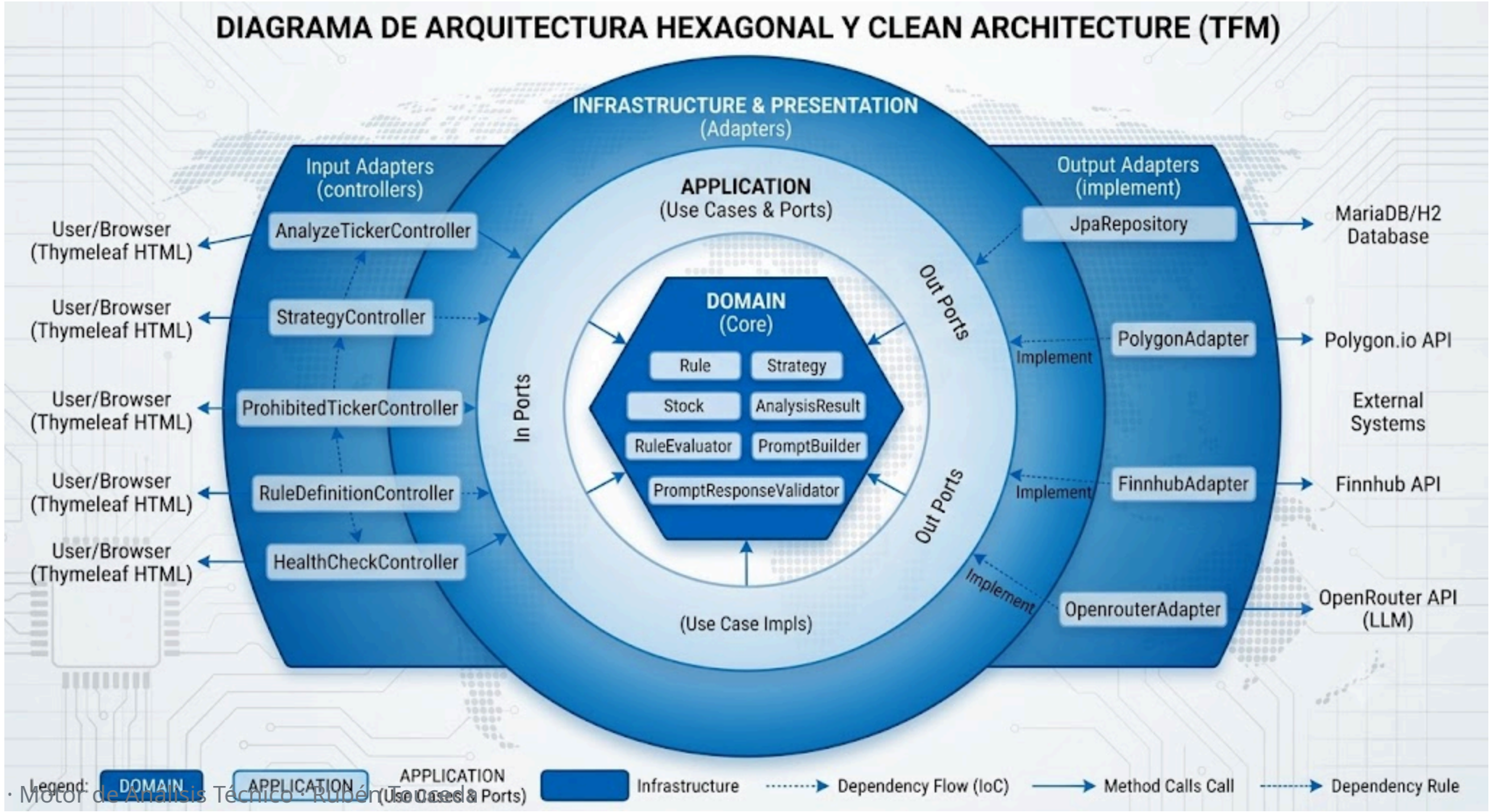
El recorrido de una acción, de candidata a evaluada:

1. Ingesta de tickers o sugerencias automáticas (scrapper propio de Finviz) con filtro de seguridad
2. Enriquecimiento: OHLCV + SMA 20/50/200 y volumen (Polygon/Finnhub)
3. Evaluación declarativa: `Precio > SMA(50)` AND `Volumen > Media(20)`
4. Cálculo de R:R y score ponderado 0-100 por estrategia
5. Análisis IA a petición (async, con rotación de modelos ante 429)
6. Persistencia y seguimiento

Motor de reglas + puntuación

- Cada **regla** es una condición simple sobre el mercado, del tipo "el precio está por encima de la media de 50 días". Responde sí o no, y explica por qué enseñando los números.
- Una **estrategia** junta varias reglas que deben cumplirse **todas a la vez**. Cada regla tiene un **peso** (su importancia) y la estrategia una **nota mínima**: el resultado es una puntuación de 0 a 100.
- El **catálogo** es la lista cerrada de indicadores disponibles, con la ficha de cada uno: qué valores acepta (la media admite 20, 50 o 200 días), cómo se calcula y dónde puede usarse.
- Actúa como guardián: si pides una media de 100 días, lo rechaza antes de calcular nada, en vez de fallar en silencio a mitad del cálculo.
- Todo queda registrado: se puede ver regla por regla cuáles se cumplieron.

Arquitectura hexagonal



Fronteras de la IA (diseño intencionado)

Motor determinista	IA interpretativa
Evalúa reglas y métricas	Redacta síntesis desde métricas
Reproducible, testeable	No decide, no valida, no señaliza
1165 tests lo cubren	Si falla → fallback + aviso

Modelo principal `laguna-s-2.1` + 4 reservas · Planes gratuitos: latencias altas **asumidas por diseño**, importa la corrección, no la velocidad.
Caso real: los 5 modelos fallaron con 429 y la evaluación siguió intacta con aviso amistoso.

Stack y calidad

- **Backend**: Java 21 · Spring Boot · JPA · WebClient · Thymeleaf + Bootstrap 5
- **Datos**: H2 (dev) · MariaDB (prod) · Finnhub / Polygon / Finviz / OpenRouter
- **Calidad**: 1165 tests, JaCoCo ≥80%, **SonarQube A** (63→0 issues), CI en PRs
- **Despliegue**: Docker + VPS propio con Nginx/SSL → https://tfm.rubentouceda.es
- Se evaluó Railway, pero la comunicación con la base de datos era lenta; el VPS además amplió conocimientos (endurecimiento, fail2ban, backups, red y certificados gestionados a mano)

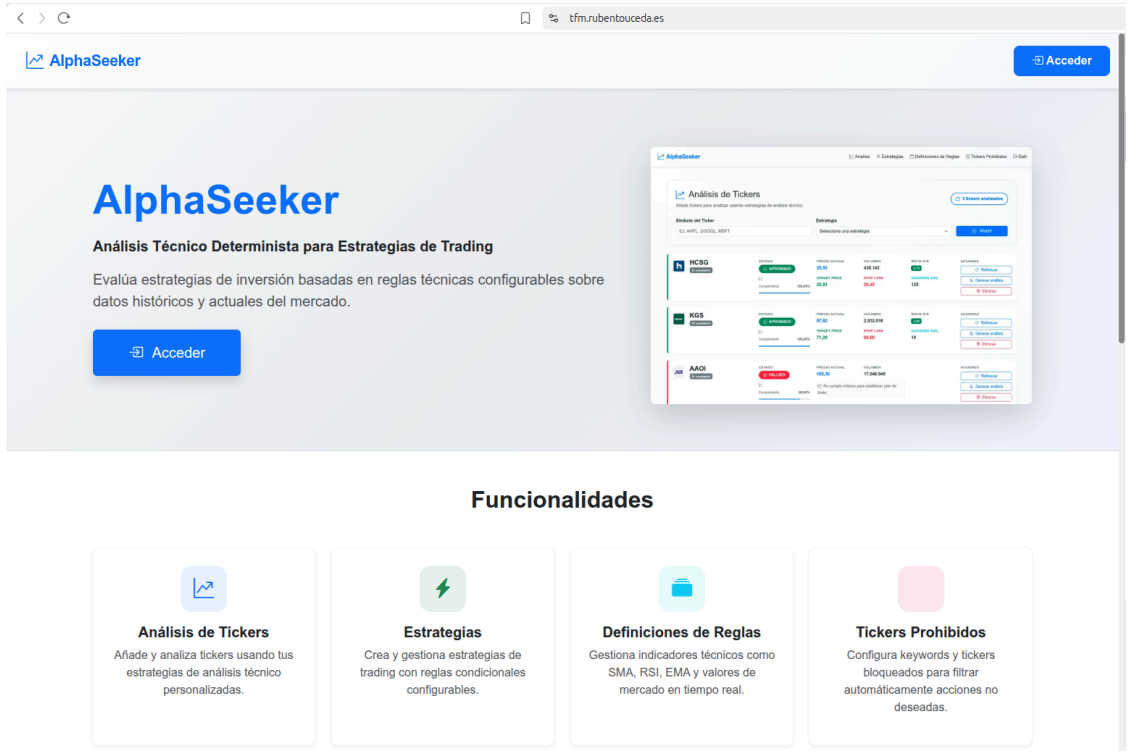
Desarrollo con IA: 2 fases + sello propio

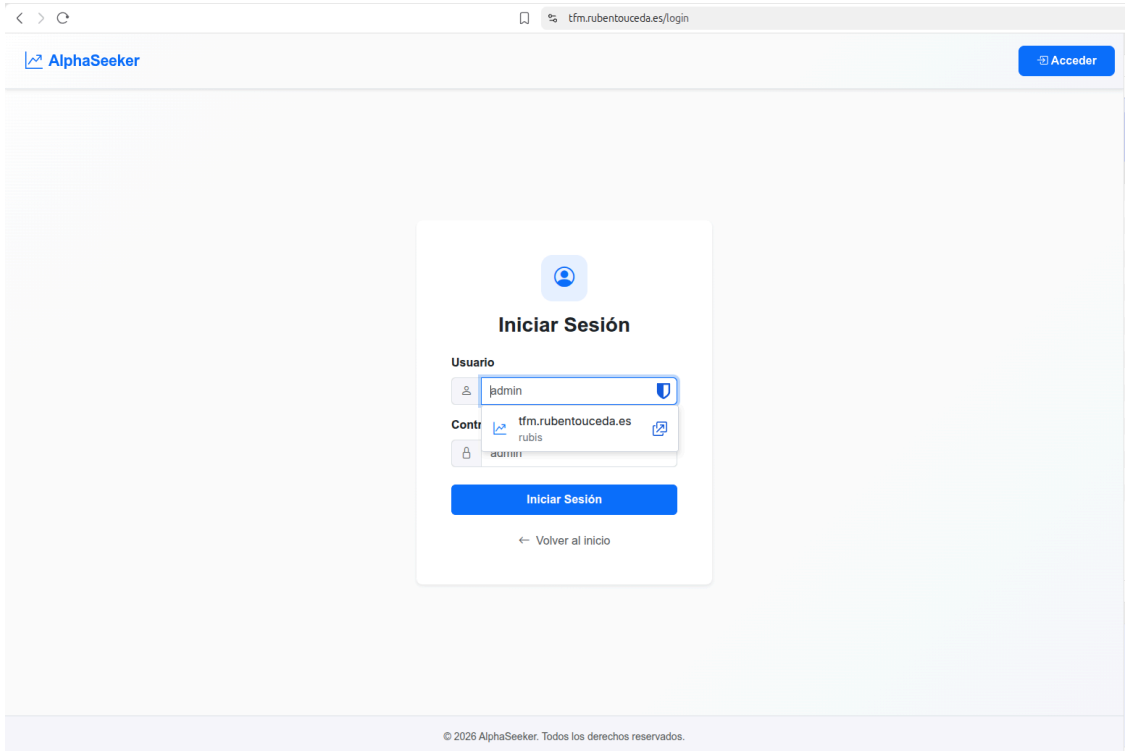
1. **Copilot**: arranque y estructura con autocompletado, siempre supervisado
2. **OpenCode**: 1 rama = 1 tema = 1 doc = 1 PR, validación con menú
   (~40 PRs en el cierre, `docs/` con trazabilidad total)

En ambas fases hubo **desarrollo propio permanente**: se modificaban las propuestas del agente, se hacían partes a mano y se resolvían imprevistos y tuning.

Frontera sagrada: **la IA propone, la persona dispone**.

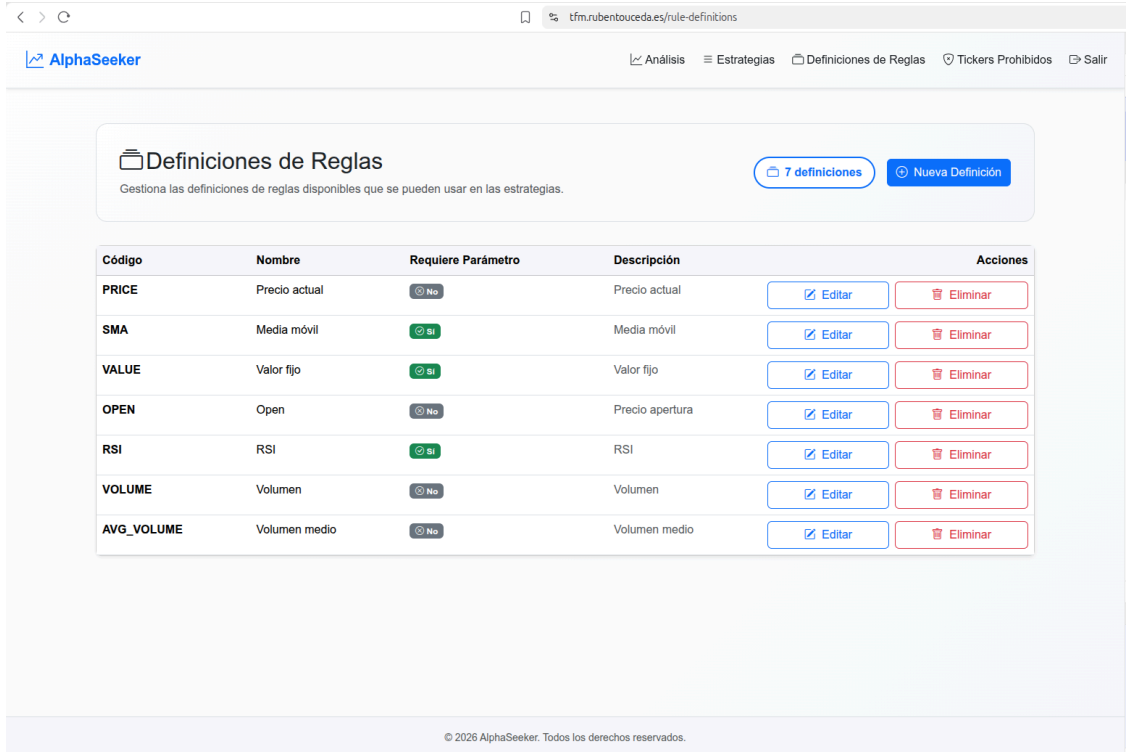
La aplicación (1/9): acceso

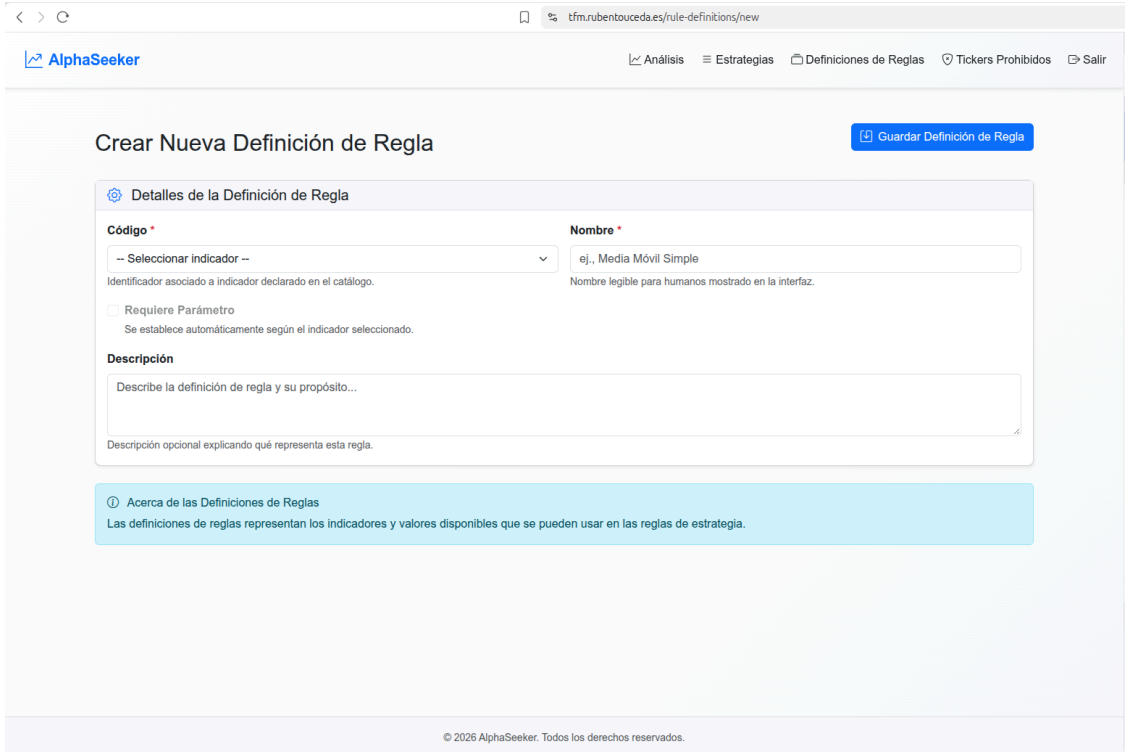




Portada pública con las 4 áreas. El resto exige login (usuario de prueba en el formulario de entrega), con bloqueo anti-fuerza bruta tras 3 fallos.

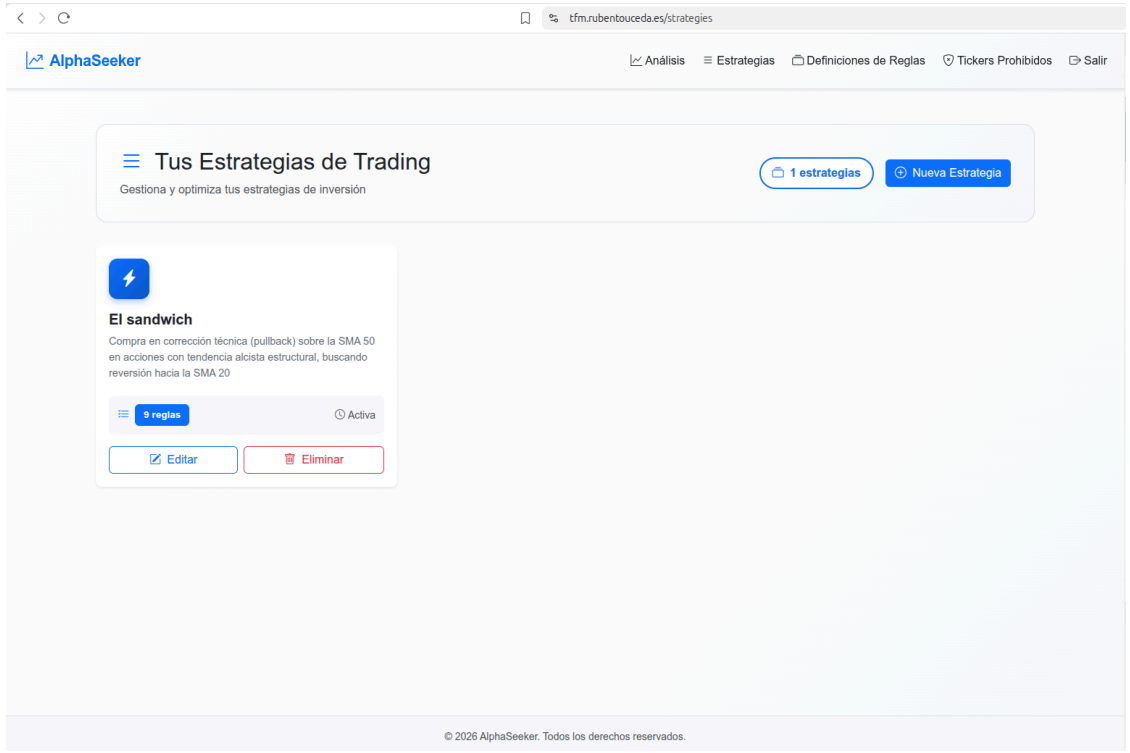
La aplicación (2/9): reglas

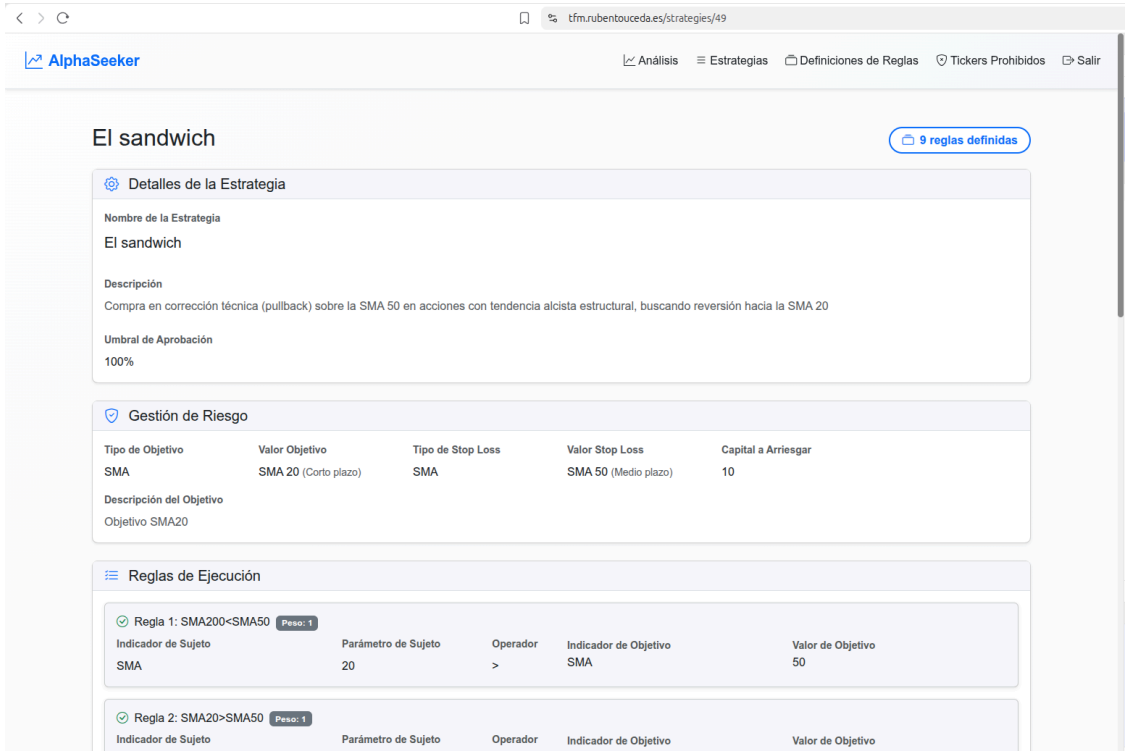




El catálogo administrable (PRICE, SMA, RSI, VOLUMEN…) dice qué parámetro necesita cada indicador. El alta es guiada: al elegir indicador, la pantalla activa o apaga el parámetro sola.

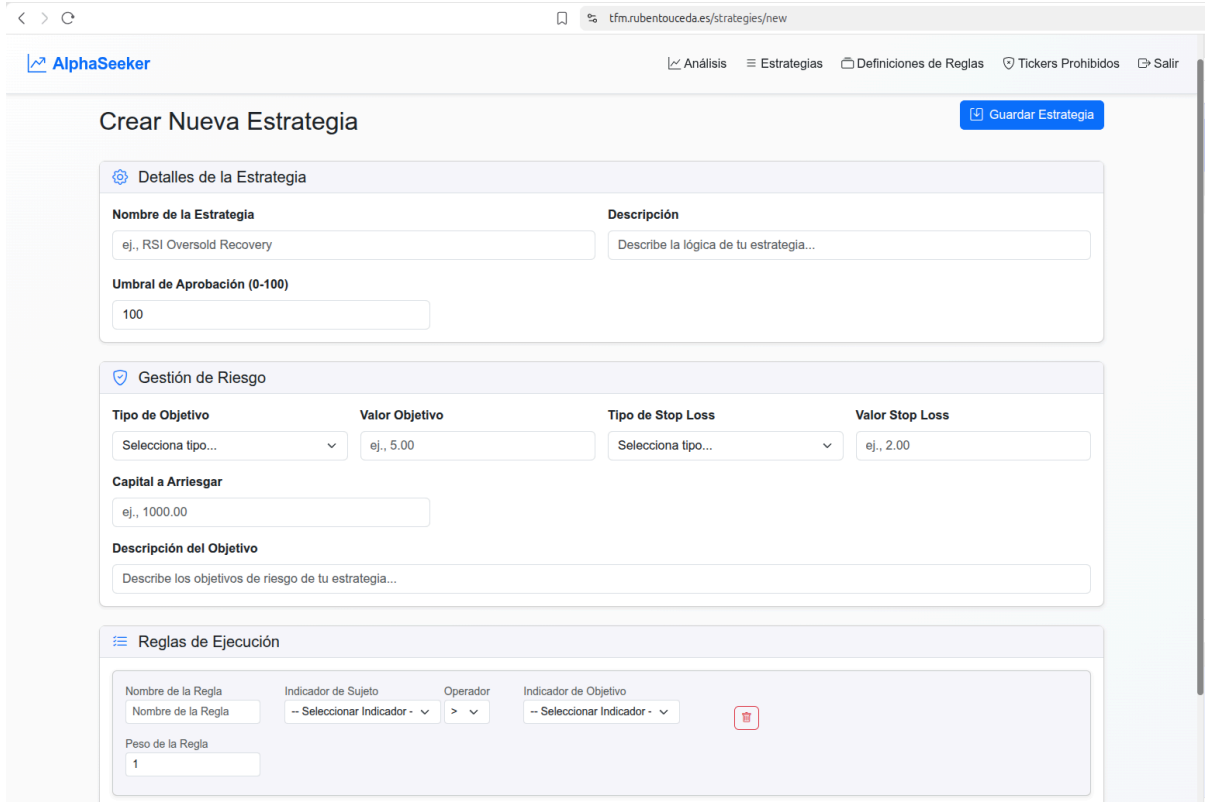
La aplicación (3/9): estrategias





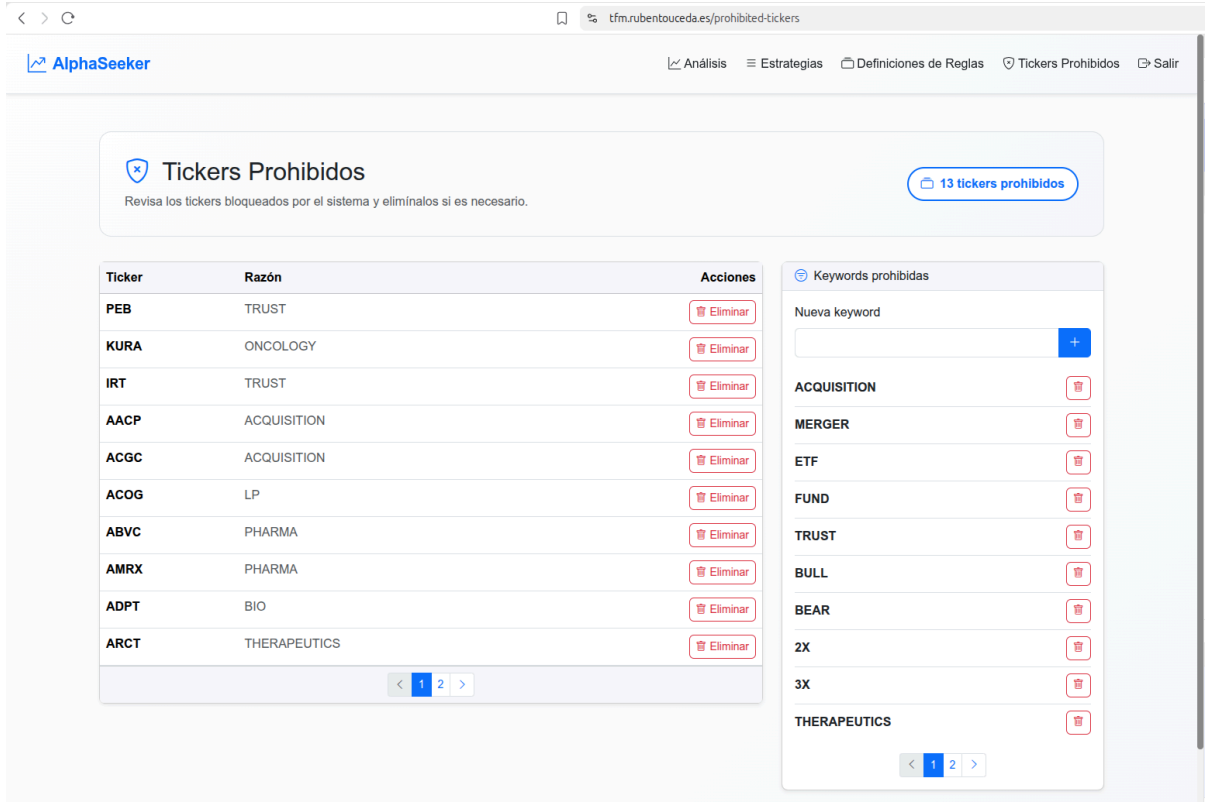
"El sandwich": compra en pullback sobre SMA50, 9 reglas, umbral 100%, con objetivo (SMA20), stop (SMA50) y capital a arriesgar definidos.

La aplicación (4/9): alta de estrategia



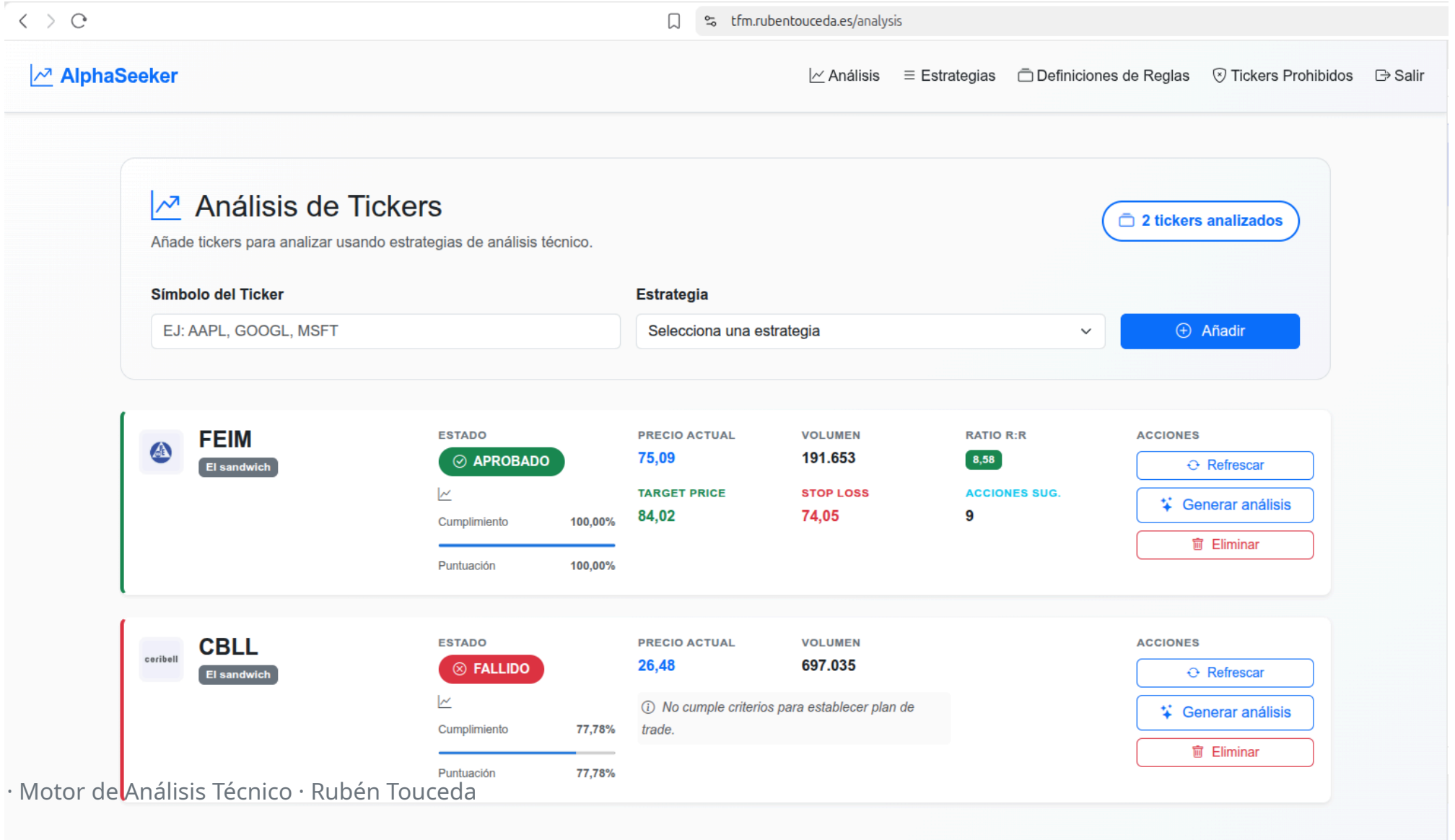
Nombre y descripción, nota mínima de aprobación (0-100), gestión de riesgo y reglas de ejecución con su peso individual.

La aplicación (5/9): filtro de seguridad

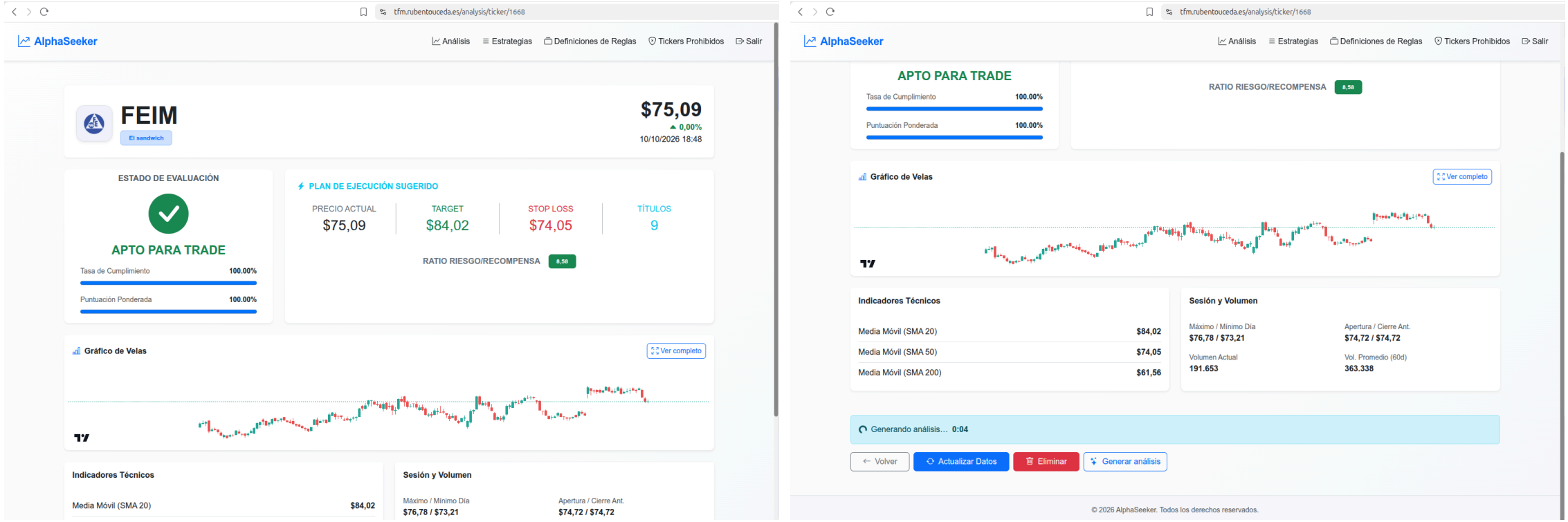


Antes de gastar un céntimo de cuota: 13 tickers vetados (trusts, farma, ETFs, apalancados 2X/3X…) y keywords gestionables que filtran solos.

La aplicación (6/9): alta y evaluación

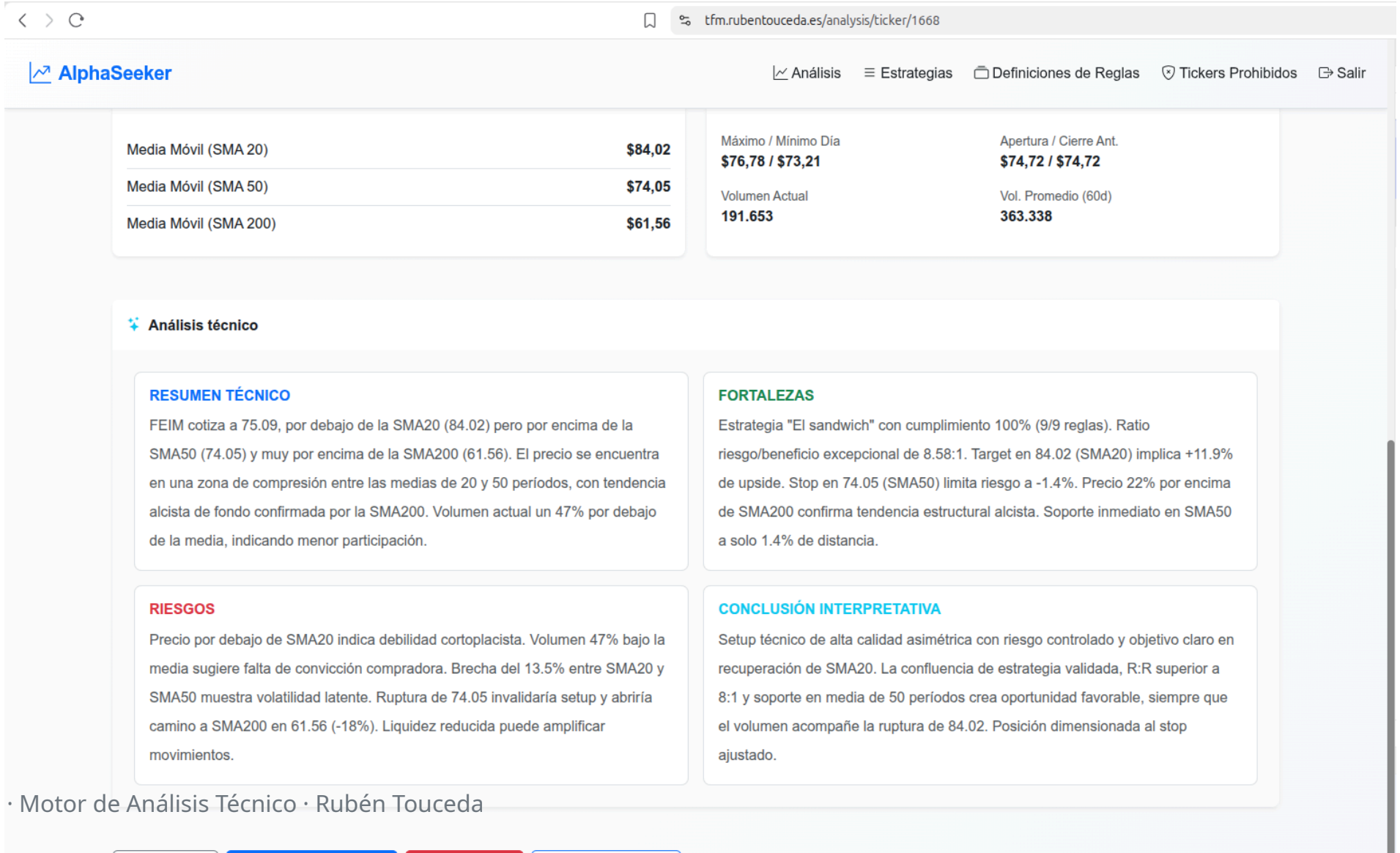


La aplicación (7/9): detalle del candidato

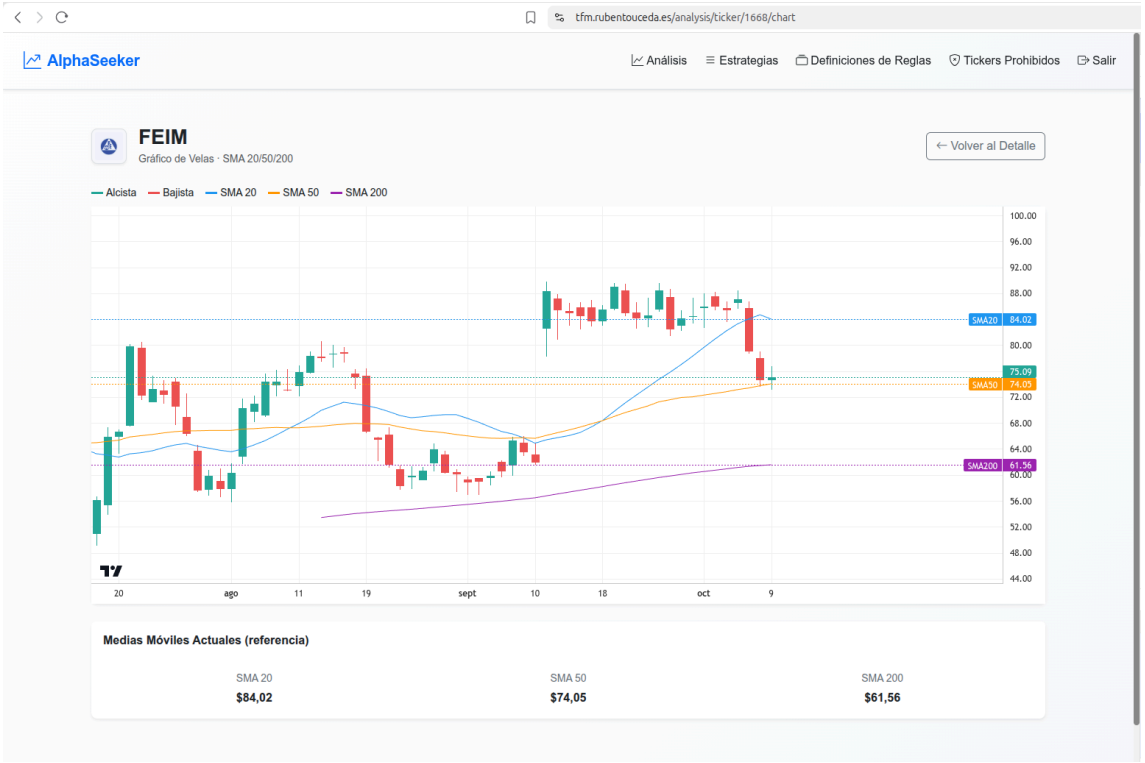


El detalle muestra veredicto, cumplimiento, plan (entrada, target, stop-loss) y velas. "Generar análisis" lanza la IA en segundo plano con banner de progreso: se puede seguir navegando.

La aplicación (8/9): síntesis IA



La aplicación (9/9): gráfico y descarte



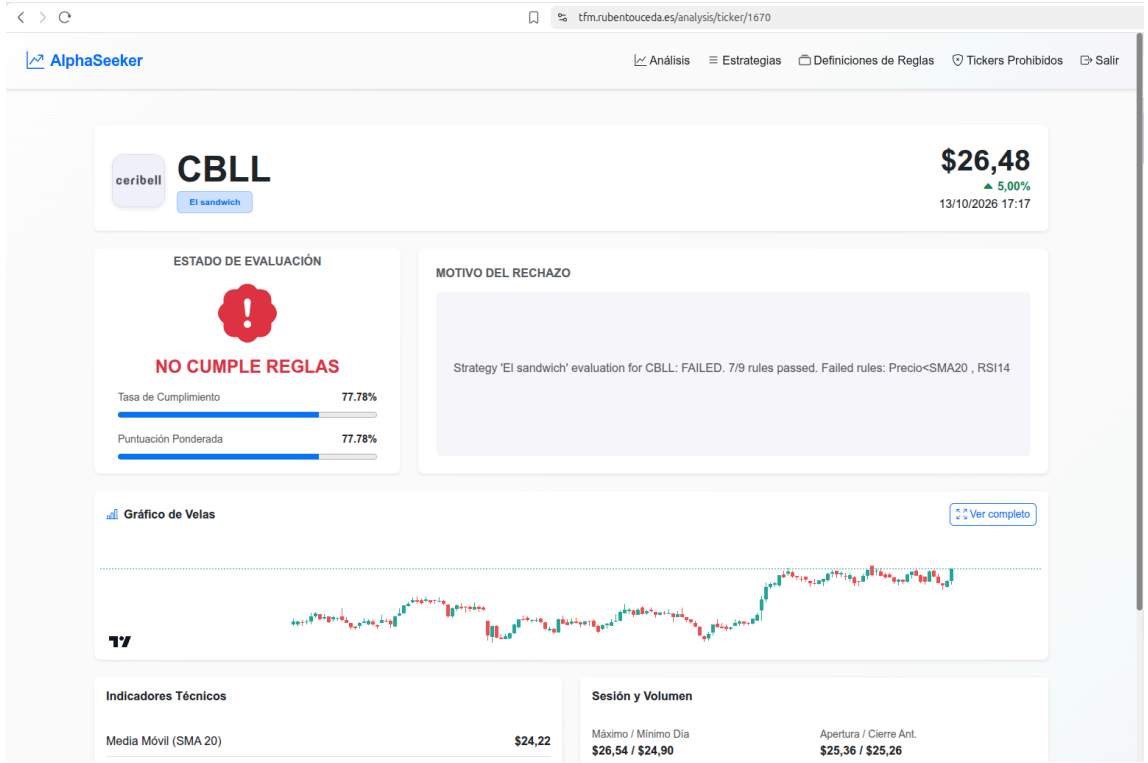


Gráfico completo con SMA 20/50/200 para el estudio visual. Y el descarte también informa: CBLL explica qué 2 reglas fallaron (Precio<SMA20, RSI14).

Limitaciones y futuro

Hoy no hace: recomendar inversión · operar · tiempo real · notificaciones

Líneas futuras:

- OpenAPI pública cuando haya API REST real
- Servicio ML separado (Python/FastAPI) desacoplado del motor
- Migración Spring Boot 4.x (fin del soporte OSS 3.5.x)

Cierre: las 5 piezas

- ✅ README que se entiende sin abrir código
- ✅ Código supervisado en GitHub
- ✅ Despliegue productivo funcionando
- ✅ Estas slides versionadas
- ⬜ Formulario final con las URLs

Gracias.